

✓ BERT

```
# =====  
# PART 1: Install & Imports  
# =====  
!pip install transformers datasets scikit-learn openpyxl -q  
  
import pandas as pd  
import numpy as np  
import torch  
from google.colab import files  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import classification_report, accuracy_score  
from transformers import (  
    BertTokenizer,  
    BertForSequenceClassification,  
    Trainer,  
    TrainingArguments,  
)  
from datasets import Dataset  
import warnings  
warnings.filterwarnings("ignore")  
  
print("✅ Libraries loaded!")  
print(f"GPU: {torch.cuda.is_available()}")
```

✅ Libraries loaded!
GPU: True

```
# =====  
# PART 2: Upload Files Manually  
# =====  
from google.colab import files  
  
print("📁 Upload: Fendi Fake Reviews")  
up = files.upload(); fendi_fake_file = next(iter(up))  
  
print("\n📁 Upload: Prada Fake Reviews")  
up = files.upload(); prada_fake_file = next(iter(up))  
  
print("\n📁 Upload: LV Reviews")
```

```

up = files.upload(); lv_file = next(iter(up))

print("\n📁 Upload: YSL Reviews")
up = files.upload(); ysl_file = next(iter(up))

print("\n📁 Upload: Chanel Fake Reviews")
up = files.upload(); chanel_fake_file = next(iter(up))

print("\n📁 Upload: Coach Reviews (mixed fake+real)")
up = files.upload(); coach_file = next(iter(up))

print("\n📁 Upload: Coach Info (to map fake/real)")
up = files.upload(); coach_info_file = next(iter(up))

print("\n✅ All files uploaded!")

```

📁 Upload: Fendi Fake Reviews
 Choose Files no files selected Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
 Saving fendi_reviews_Fake (1).xlsx to fendi_reviews_Fake (1) (1).xlsx

📁 Upload: Prada Fake Reviews
 Choose Files no files selected Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
 Saving prada reviwex.xlsx to prada reviwex (1).xlsx

📁 Upload: LV Reviews
 Choose Files no files selected Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
 Saving reviewLV.xlsx to reviewLV (1).xlsx

📁 Upload: YSL Reviews
 Choose Files no files selected Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
 Saving YSL reviews (1).xlsx to YSL reviews (1) (1).xlsx

📁 Upload: Chanel Fake Reviews
 Choose Files no files selected Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
 Saving chanel reviews .xlsx to chanel reviews (1).xlsx

📁 Upload: Coach Reviews (mixed fake+real)
 Choose Files no files selected Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
 Saving reviews of coach (2).xlsx to reviews of coach (2) (1).xlsx

```

import numpy as np
import torch
import gc
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, classification_report
from datasets import Dataset
from transformers import (
    BertTokenizer,
    BertForSequenceClassification,
    TrainingArguments,
    Trainer,
)

# — Settings —————
MODEL_NAME = "bert-base-uncased"
N_SPLITS    = 5
SEED        = 42
MAX_LENGTH  = 256

print(f"Model      : {MODEL_NAME}")
print(f"Folds       : {N_SPLITS}")
print(f"Max length  : {MAX_LENGTH}")
print(f"GPU          : {torch.cuda.is_available()}")

# — Load tokenizer & prepare data —————
tokenizer = BertTokenizer.from_pretrained(MODEL_NAME)

texts = df_all["review"].tolist()
labels = df_all["label"].tolist()

print(f"Total samples : {len(texts)}")
print(f"Label dist     : {dict(zip(*np.unique(labels, return_counts=True)))}")

```

```

Model      : bert-base-uncased
Folds      : 5
Max length : 256
GPU        : True
Total samples : 1458
Label dist  : {np.int64(0): np.int64(1118), np.int64(1): np.int64(340)}

```

```

# =====
# PART 3: Load & Preprocess
# =====

```

```

BAD = ['nan', 'no_review', 'none', '']

def clean(df, text_col, label):
    df = df[[df.columns[0], text_col]].copy()
    df.columns = ['id', 'review']
    df = df.dropna(subset=['review'])
    df = df[~df['review'].astype(str).str.strip().str.lower().isin(BAD)]
    df['label'] = label
    return df[['review', 'label']]

# — Fendi Fake → label=0
df_fendi = pd.read_excel(fendi_fake_file)
df_fendi = clean(df_fendi, df_fendi.columns[1], label=0)
df_fendi['brand'] = 'Fendi'

# — Prada Fake → label=0
df_prada = pd.read_excel(prada_fake_file)
df_prada = clean(df_prada, df_prada.columns[1], label=0)
df_prada['brand'] = 'Prada'

# — LV → label=0
df_lv = pd.read_excel(lv_file)
df_lv = clean(df_lv, df_lv.columns[1], label=0)
df_lv['brand'] = 'LV'

# — YSL → label=0
df_ysl = pd.read_excel(ysl_file)
df_ysl = clean(df_ysl, df_ysl.columns[1], label=0)
df_ysl['brand'] = 'YSL'

# — Chanel Fake → label=0
df_chanel = pd.read_excel(chanel_fake_file)
df_chanel = clean(df_chanel, df_chanel.columns[1], label=0)
df_chanel['brand'] = 'Chanel'

# — Coach: map fake/real from info file
df_coach_r = pd.read_excel(coach_file)
df_coach_r.columns = ['id', 'review']

df_coach_i = pd.read_excel(coach_info_file)
df_coach_i.columns = [c.strip().lower().replace(' ', '_') for c in df_coach_i.columns]
auth_map = dict(zip(df_coach_i['id'].astype(int), df_coach_i['fake/real']))

df_coach_r['label'] = df_coach_r['id'].astype(int).map(auth_map)

```

```
df_coach_r = df_coach_r.dropna(subset=['review', 'label'])
df_coach_r = df_coach_r[~df_coach_r['review'].astype(str).str.!.]
df_coach_r['label'] = df_coach_r['label'].astype(int)
df_coach = df_coach_r[['review', 'label']]
df_coach['brand'] = 'Coach'

# — Merge All
df_all = pd.concat([
    df_fendi, df_prada, df_lv, df_ysl, df_chanel, df_coach
], ignore_index=True)

df_all['review'] = df_all['review'].astype(str).str.strip()
df_all.drop_duplicates(inplace=True)
df_all.reset_index(drop=True, inplace=True)

print(f"✅ Total reviews: {len(df_all)}")
print(df_all['label'].value_counts().rename({0: 'Fake', 1: 'Real'}
```

```
✅ Total reviews: 1468
label
Fake      1128
Real       340
Name: count, dtype: int64
```

```
# —Helper functions —————
def tokenize(batch):
    return tokenizer(
        batch["text"],
        padding="max_length",
        truncation=True,
        max_length=MAX_LENGTH,
    )

def make_hf_dataset(texts, labels):
    ds = Dataset.from_dict({"text": texts, "label": labels})
    return ds.map(tokenize, batched=True)

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {"accuracy": accuracy_score(labels, preds)}

def get_training_args(output_dir):
    return TrainingArguments(
        output_dir=output_dir,
        eval_strategy="epoch",
        save_strategy="no",
        num_train_epochs=3,
        per_device_train_batch_size=16,
        per_device_eval_batch_size=32,
        learning_rate=2e-5,
        weight_decay=0.01,
        load_best_model_at_end=False,
        logging_steps=50,
        fp16=torch.cuda.is_available(),
        report_to="none",
    )

print("✅ Helper functions ready!")
```

✅ Helper functions ready!

```
# — Run K-Fold Cross Validation —————
skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True)
bert_results = []

for fold, (train_idx, val_idx) in enumerate(skf.split(texts, labels)):
    print(f"\n{'-'*50}")
```

```

print(f"  BERT – Fold {fold}/{N_SPLITS}")
print(f"{'-'*50}")

train_texts = [texts[i] for i in train_idx]
train_labels = [labels[i] for i in train_idx]
val_texts = [texts[i] for i in val_idx]
val_labels = [labels[i] for i in val_idx]
print(f"  Train: {len(train_texts)} | Val: {len(val_texts)}")

train_ds = make_hf_dataset(train_texts, train_labels)
val_ds = make_hf_dataset(val_texts, val_labels)

model = BertForSequenceClassification.from_pretrained(MODEL_PATH)
trainer = Trainer(
    model=model,
    args=get_training_args(f"./bert_fold{fold}"),
    train_dataset=train_ds,
    eval_dataset=val_ds,
    compute_metrics=compute_metrics,
)

trainer.train()

preds_out = trainer.predict(val_ds)
y_pred = np.argmax(preds_out.predictions, axis=-1)
acc = accuracy_score(val_labels, y_pred)
print(f"\n  🟢 Fold {fold} Accuracy: {acc:.4f}")

bert_results.append({
    "fold": fold,
    "accuracy": acc,
    "y_true": val_labels,
    "y_pred": y_pred.tolist(),
})

del model, trainer, train_ds, val_ds
gc.collect()
if torch.cuda.is_available():
    torch.cuda.empty_cache()

print(f"\n  🎉 BERT Cross-Validation done!")

```

BERT – Fold 1/5

```
Train: 1166 | Val: 292
Map: 100% 1166/1166 [00:00<00:00, 2258.90 examples/s]

Map: 100% 292/292 [00:00<00:00, 1151.43 examples/s]

Loading weights: 100% 199/199 [00:00<00:00, 215.65it/s, Materializing param=bert.pooler.dense]

BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.dense.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |

Notes:
- UNEXPECTED :can be ignored when loading from different task/ai
- MISSING :those params were newly initialized because missing
[219/219 00:47, Epoch 3/3]
```

Epoch	Training Loss	Validation Loss	Accuracy
1	0.448609	0.306666	0.873288
2	0.280930	0.292287	0.897260
3	0.125390	0.302976	0.904110

 Fold 1 Accuracy: 0.9041

```
BERT - Fold 2/5

Train: 1166 | Val: 292
Map: 100% 1166/1166 [00:00<00:00, 3123.12 examples/s]

Map: 100% 292/292 [00:00<00:00, 2905.61 examples/s]

Loading weights: 100% 199/199 [00:00<00:00, 544.80it/s, Materializing param=bert.pooler.dense]

BertForSequenceClassification LOAD REPORT from: bert-base-uncased
```


Key	Status
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/ai
- MISSING :those params were newly initialized because missin

[219/219 00:44, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy
1	0.459537	0.281904	0.866438
2	0.268620	0.261030	0.886986
3	0.120546	0.282986	0.893836



Fold 2 Accuracy: 0.8938

BERT — Fold 3/5

Train: 1166 | Val: 292

Map: 100% 1166/1166 [00:00<00:00, 3634.90 examples/s]

Map: 100% 292/292 [00:00<00:00, 2894.45 examples/s]

Loading weights: 100% 199/199 [00:00<00:00, 609.42it/s, Materializing param=bert.pooler.dense

BertForSequenceClassification LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

Notes:

- UNEXPECTED :can be ignored when loading from different task/ai
 - MISSING :those params were newly initialized because missing
- [219/219 00:44, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy
1	0.453109	0.292211	0.886986
2	0.276375	0.295367	0.876712
3	0.128975	0.301578	0.893836



Fold 3 Accuracy: 0.8938

BERT - Fold 4/5

Train: 1167 | Val: 291

Map: 100% 1167/1167 [00:00<00:00, 3597.69 examples/s]

Map: 100% 291/291 [00:00<00:00, 2186.24 examples/s]

Loading weights: 100% 199/199 [00:00<00:00, 564.72it/s, Materializing param=bert.pooler.dense]

BertForSequenceClassification LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/ai
 - MISSING :those params were newly initialized because missing
- [219/219 00:45, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy
1	0.463813	0.301845	0.886598
2	0.248943	0.212310	0.917526
3	0.112435	0.199852	0.924399

✅ Fold 4 Accuracy: 0.9244

BERT — Fold 5/5

Train: 1167 | Val: 291

Map: 100% 1167/1167 [00:00<00:00, 3340.55 examples/s]

Map: 100% 291/291 [00:00<00:00, 2738.50 examples/s]

Loading weights: 100% 199/199 [00:00<00:00, 574.78it/s, Materializing param=bert.pooler.dense

BertForSequenceClassification LOAD REPORT from: bert-base-uncased

Key	Status
-----	--------

cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED

```
# — Accuracy per fold —
print("=" * 55)
print("📊 BERT — Accuracy per Fold")
print("=" * 55)
```

```
accs = [r["accuracy"] for r in bert_results]

for r in bert_results:
    print(f"  Fold {r['fold']}: {r['accuracy']:.4f}")

print(f"\n Mean : {np.mean(accs):.4f}")
print(f" Std  : {np.std(accs):.4f}")
print(f" Min  : {np.min(accs):.4f}")
print(f" Max  : {np.max(accs):.4f}")
```

```
# — Cell 7: Overall Classification Report —
all_y_true = [y for r in bert_results for y in r["y_true"]]
all_y_pred = [y for r in bert_results for y in r["y_pred"]]

report = classification_report(
    all_y_true, all_y_pred,
```

```

target_names=["Fake", "Real"],
digits=4,
output_dict=True,
)

print("=" * 55)
print("📊 BERT – Classification Report (All Folds)")
print("=" * 55)
print(f"\n  {'Class':<12} {'Precision':>10} {'Recall':>10} {'F1-Score':>10} {'Support':>10}")
print(f"  {'-'*52}")
print(f"  {'Fake':<12} {report['Fake']['precision']:>10.4f} {report['Fake']['recall']:>10.4f} {report['Fake']['f1_score']:>10.4f} {report['Fake']['support']:>10.4f}")
print(f"  {'Real':<12} {report['Real']['precision']:>10.4f} {report['Real']['recall']:>10.4f} {report['Real']['f1_score']:>10.4f} {report['Real']['support']:>10.4f}")
print(f"  {'-'*52}")
print(f"  {'Accuracy':<12} {'':>10} {'':>10} {report['accuracy']:>10.4f} {report['accuracy']:>10.4f}")
print(f"  {'Macro Avg':<12} {report['macro avg']['precision']:>10.4f} {report['macro avg']['recall']:>10.4f} {report['macro avg']['f1_score']:>10.4f} {report['macro avg']['support']:>10.4f}")
print(f"  {'Weighted Avg':<12} {report['weighted avg']['precision']:>10.4f} {report['weighted avg']['recall']:>10.4f} {report['weighted avg']['f1_score']:>10.4f} {report['weighted avg']['support']:>10.4f}")

print(f"\n  ——— Per-Class Accuracy (Recall) ———")
print(f"  Fake Accuracy : {report['Fake']['recall']:.4f} ({report['Fake']['recall']:.4f})")
print(f"  Real Accuracy : {report['Real']['recall']:.4f} ({report['Real']['recall']:.4f})")
print(f"  Overall Acc   : {report['accuracy']:.4f} ({report['accuracy']:.4f})")

```

📊 BERT – Accuracy per Fold

```

Fold 1: 0.9041
Fold 2: 0.8938
Fold 3: 0.8938
Fold 4: 0.9244
Fold 5: 0.9210

```

```

Mean : 0.9074
Std  : 0.0131
Min  : 0.8938
Max  : 0.9244

```

📊 BERT – Classification Report (All Folds)

Class	Precision	Recall	F1-Score	Support
Fake	0.9323	0.9481	0.9401	1118
Real	0.8193	0.7735	0.7958	340
Accuracy			0.9074	1458
Macro Avg	0.8758	0.8608	0.8679	1458
Weighted Avg	0.9059	0.9074	0.9065	1458

—— Per-Class Accuracy (Recall) ——
 Fake Accuracy : 0.9481 (94.81%)
 Real Accuracy : 0.7735 (77.35%)
 Overall Acc : 0.9074 (90.74%)

```
# — Confusion Matrix —————
cm = confusion_matrix(all_y_true, all_y_pred)

print("=" * 55)
print("📊 BERT – Confusion Matrix (All Folds)")
print("=" * 55)
print(f"\n                Predicted")
print(f"                Fake      Real")
print(f"Actual  Fake  {cm[0][0]:>6}    {cm[0][1]:>6}    | Total")
print(f"        Real  {cm[1][0]:>6}    {cm[1][1]:>6}    | Total")
print(f"        _____")
print(f"                Total {cm[0][0]+cm[1][0]:>6}    {cm[0][1]+cm[1][1]:>6}")
print(f"\n✅ Correct Fake : {cm[0][0]} ❌ Wrong Fake : {cm[0][1]}")
print(f"✅ Correct Real : {cm[1][1]} ❌ Wrong Real : {cm[1][0]}")
```

```
=====
📊 BERT – Confusion Matrix (All Folds)
=====
```

		Predicted		
		Fake	Real	
Actual	Fake	1060	58	Total: 1118
	Real	77	263	
Total		1137	321	

✅ Correct Fake : 1060 ❌ Wrong Fake : 58
 ✅ Correct Real : 263 ❌ Wrong Real : 77

▼ RoBERTa

```
# —Imports —————
import numpy as np
import torch
import gc
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, classification_report
from datasets import Dataset
from transformers import (
    RobertaTokenizer,
    RobertaForSequenceClassification,
    TrainingArguments,
    Trainer,
)
```

```
# — Settings —————
MODEL_NAME = "roberta-base"
N_SPLITS    = 5
SEED        = 42
MAX_LENGTH  = 256

print(f"Model      : {MODEL_NAME}")
print(f"Folds      : {N_SPLITS}")
print(f"Max length : {MAX_LENGTH}")
print(f"GPU         : {torch.cuda.is_available()}")

# — Load tokenizer & prepare data —————
tokenizer = RobertaTokenizer.from_pretrained(MODEL_NAME)

texts = df_all["review"].tolist()
labels = df_all["label"].tolist()

print(f"Total samples : {len(texts)}")
print(f"Label dist      : {dict(zip(*np.unique(labels, return_counts=True)))}")
```

```
Model      : roberta-base
Folds      : 5
Max length : 256
GPU        : True
Total samples : 1458
Label dist   : {np.int64(0): np.int64(1118), np.int64(1): np.int64(340)}
```

```
# — Helper functions —————
def tokenize(batch):
    return tokenizer(
        batch["text"],
        padding="max_length",
        truncation=True,
        max_length=MAX_LENGTH,
    )

def make_hf_dataset(texts, labels):
    ds = Dataset.from_dict({"text": texts, "label": labels})
    return ds.map(tokenize, batched=True)

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {"accuracy": accuracy_score(labels, preds)}

def get_training_args(output_dir):
    return TrainingArguments(
        output_dir=output_dir,
        eval_strategy="epoch",
        save_strategy="no",
        num_train_epochs=3,
        per_device_train_batch_size=16,
        per_device_eval_batch_size=32,
        learning_rate=2e-5,
        weight_decay=0.01,
        load_best_model_at_end=False,
        logging_steps=50,
        fp16=torch.cuda.is_available(),
        report_to="none",
    )

print("✅ Helper functions ready!")
```

✅ Helper functions ready!

```
# —Run K-Fold Cross Validation —————
skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True)
roberta_results = []

for fold, (train_idx, val_idx) in enumerate(skf.split(texts, labels)):
    print(f"\n{'-'*50}")
```

```
print(f"  RoBERTa – Fold {fold}/{N_SPLITS}")
print(f"{'-'*50}")

train_texts = [texts[i] for i in train_idx]
train_labels = [labels[i] for i in train_idx]
val_texts = [texts[i] for i in val_idx]
val_labels = [labels[i] for i in val_idx]
print(f"  Train: {len(train_texts)} | Val: {len(val_texts)}")

train_ds = make_hf_dataset(train_texts, train_labels)
val_ds = make_hf_dataset(val_texts, val_labels)

model = RobertaForSequenceClassification.from_pretrained
trainer = Trainer(
    model=model,
    args=get_training_args(f"./roberta_fold{fold}"),
    train_dataset=train_ds,
    eval_dataset=val_ds,
    compute_metrics=compute_metrics,
)

trainer.train()

preds_out = trainer.predict(val_ds)
y_pred = np.argmax(preds_out.predictions, axis=-1)
acc = accuracy_score(val_labels, y_pred)
print(f"\n 🟢 Fold {fold} Accuracy: {acc:.4f}")

roberta_results.append({
    "fold": fold,
    "accuracy": acc,
    "y_true": val_labels,
    "y_pred": y_pred.tolist(),
})

del model, trainer, train_ds, val_ds
gc.collect()
if torch.cuda.is_available():
    torch.cuda.empty_cache()

print("\n 🎉 RoBERTa Cross-Validation done!")
```


RoBERTa — Fold 1/5

Train: 1166 | Val: 292

Map: 100%1166/1166 [00:00<00:00, 1438.15 examples/s]

Map: 100%292/292 [00:00<00:00, 1596.35 examples/s]

config.json: 100%481/481 [00:00<00:00, 14.4kB/s]

model.safetensors: 100%499M/499M [00:04<00:00, 137MB/s]

Loading weights: 100% 197/197 [00:00<00:00, 434.62it/s, Materializing param=roberta.encoder

RobertaForSequenceClassification LOAD REPORT from: roberta-base		
Key	Status	
-----+-----+		
lm_head.dense.weight	UNEXPECTED	
lm_head.layer_norm.weight	UNEXPECTED	
lm_head.bias	UNEXPECTED	
lm_head.layer_norm.bias	UNEXPECTED	
roberta.embeddings.position_ids	UNEXPECTED	
lm_head.dense.bias	UNEXPECTED	
classifier.out_proj.bias	MISSING	
classifier.dense.weight	MISSING	
classifier.dense.bias	MISSING	
classifier.out_proj.weight	MISSING	

Notes:

- UNEXPECTED :can be ignored when loading from different task/ai
- MISSING :those params were newly initialized because missi

[219/219 00:42, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy
1	0.522072	0.280272	0.886986
2	0.343201	0.257483	0.886986
3	0.173691	0.273318	0.893836

 Fold 1 Accuracy: 0.8938

RoBERTa — Fold 2/5

Train: 1166 | Val: 292

Map: 100%1166/1166 [00:00<00:00, 4321.86 examples/s]

Map: 100%292/292 [00:00<00:00, 2318.44 examples/

s]

```

# — Accuracy per fold —————
print("=" * 55)
print("🇩🇪 RoBERTa – Accuracy per Fold")
print("=" * 55)

accs = [r["accuracy"] for r in roberta_results]

for r in roberta_results:
    print(f"  Fold {r['fold']}: {r['accuracy']:.4f}")

print(f"\n  Mean : {np.mean(accs):.4f}")
print(f"  Std  : {np.std(accs):.4f}")
print(f"  Min  : {np.min(accs):.4f}")
print(f"  Max  : {np.max(accs):.4f}")

# — Overall Classification Report —————
all_y_true = [y for r in roberta_results for y in r["y_true"]]
all_y_pred = [y for r in roberta_results for y in r["y_pred"]]

report = classification_report(
    all_y_true, all_y_pred,
    target_names=["Fake", "Real"],
    digits=4,
    output_dict=True,
)

print("=" * 55)
print("📄 RoBERTa – Classification Report (All Folds)")
print("=" * 55)
print(f"\n  {'Class':<12} {'Precision':>10} {'Recall':>10} {'F1':>10}")
print(f"  {'-'*52}")
print(f"  {'Fake':<12} {report['Fake']['precision']:>10.4f} {report['Fake']['recall']:>10.4f} {report['Fake']['f1']:>10.4f}")
print(f"  {'Real':<12} {report['Real']['precision']:>10.4f} {report['Real']['recall']:>10.4f} {report['Real']['f1']:>10.4f}")
print(f"  {'-'*52}")
print(f"  {'Accuracy':<12} {'':>10} {'':>10} {report['accuracy']:>10.4f}")
print(f"  {'Macro Avg':<12} {report['macro avg']['precision']:>10.4f} {report['macro avg']['recall']:>10.4f} {report['macro avg']['f1']:>10.4f}")
print(f"  {'Weighted Avg':<12} {report['weighted avg']['precision']:>10.4f} {report['weighted avg']['recall']:>10.4f} {report['weighted avg']['f1']:>10.4f}")

print(f"\n  ——— Per-Class Accuracy (Recall) ———")
print(f"  Fake Accuracy : {report['Fake']['recall']:.4f} ({report['Fake']['precision']:.4f})")
print(f"  Real Accuracy : {report['Real']['recall']:.4f} ({report['Real']['precision']:.4f})")

```

```
print(f" Overall Acc : {report['accuracy']:.4f} ({report[''
```

```
classfier.dense.weight=====MISSING=====
classfier.dense.bias=====MISSING=====
classfier.out_proj.weight=====MISSING=====
```

Fold 1: 0.8938

Fold 2: 0.8836

- UNEXPECTED: can be ignored when loading from different task/ai
- MISSING: those params were newly initialized because missir

Fold 5: 0.9141 [219/219 00:43, Epoch 3/3]

Epoch Training Loss Validation Loss Accuracy

Mean : 0.8971

Std₁ : 0.0101 0.524315 0.289276 0.869863

Min : 0.8836

Max₂ : 0.9141 0.299346 0.280716 0.880137

 Roberta - Classification Report for All Folds

 Fold 3 Accuracy: 0.9007 Precision Recall F1-Score Support

Fake 0.9231 0.9445 0.9337 1118

Real 0.8025 0.7412 0.7706 340

Accuracy 1167 | Val: 291 0.8971 1458

Macro Avg 0.8628 0.8429 0.8523 1167/1167 [00:00<00:00, 457820 examples/

Weighted Avg 0.8950 0.8971 0.8957 1458

Per-Class Accuracy (Recall)
Fake Accuracy : 0.9445 (94.45%) 291/291 [00:00<00:00, 3251.44 examples/

Real Accuracy : 0.7412 (74.12%) s]

Overall Acc : 0.8971 (89.71%) s]

Loading weights: 100% 197/197 [00:00<00:00, 573.36it/s, Materializing param=roberta.encoder

RobertaForSequenceClassification LOAD REPORT from: roberta-base

Key	Status
lm_head.dense.weight	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.bias	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
roberta.embeddings.position_ids	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
classifier.out_proj.bias	MISSING
classifier.dense.weight	MISSING
classifier.dense.bias	MISSING
classifier.out_proj.weight	MISSING

Notes:

- UNEXPECTED : can be ignored when loading from different task/ai
- MISSING : those params were newly initialized because missir

[219/219 00:44, Epoch 3/3]

```
# —Confusion Matrix —————
cm = confusion_matrix(all_y_true, all_y_pred)

print("=" * 55)
print("📊 RoBERTa – Confusion Matrix (All Folds)")
print("=" * 55)
print(f"\n                Predicted")
print(f"                Fake      Real")
print(f"Actual  Fake  {cm[0][0]:>6}  {cm[0][1]:>6}  | Total")
print(f"        Real  {cm[1][0]:>6}  {cm[1][1]:>6}  | Total")
print(f"        _____")
print(f"        Total {cm[0][0]+cm[1][0]:>6}  {cm[0][1]+cm[1][1]:>6}")
print(f"\n✅ Correct Fake : {cm[0][0]} ❌ Wrong Fake : {cm[0][1]}")
print(f"✅ Correct Real : {cm[1][1]} ❌ Wrong Real : {cm[1][0]}")
```

Map: 100% 291/291 [00:00<00:00, 2886.27 examples/

📊 RoBERTa – Confusion Matrix (All Folds)

Loading weights: 100% 197/197 [00:00<00:00, 498.73it/s, Materializing param=roberta.encoder

		Predicted		
		Fake	Real	
Actual	Fake	1056	62	Total: 1118
	Real	88	252	Total: 340

lm_head.dense.weight	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.bias	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
roberta.embeddings.position_ids	UNEXPECTED

lm_head.dense.bias	UNEXPECTED
classifier.out_proj.bias	MISSING
classifier.dense.weight	MISSING

```
reviews_by_brand_label = (  
    df_all.groupby(["brand", "label"])["review"]  
        .count()  
        .reset_index()  
)  
  
reviews_by_brand_label["label"] = reviews_by_brand_label["label"]  
    .map(  
        0: "Fake",  
        1: "Real"  
    )  
  
print(reviews_by_brand_label)
```

	brand	label	review
0	Chanel	Fake	320
1	Coach	Fake	262
2	Coach	Real	340
3	Fendi	Fake	101
4	LV	Fake	49
5	Prada	Fake	64
6	YSL	Fake	332